Claude efficiency: measure quality, caching, batches and model choice

Researched 2026-10-12.

What the reel contributes

The five tips are caching, batching, matching models to tasks, advisor/executor composition and monitoring. They are useful starting points, but the reel supplies no reproducible benchmark. The comment-gated cheat sheet is not needed to start.

Current documentation and corrections

Use Anthropic's optimization guide (https://platform.claude.com/docs/en/about-claude/models/optimizing-for-cost-and-intelligence) as the checklist. Build a baseline and evaluation before replacing models. Batch processing (https://platform.claude.com/docs/en/build-with-claude/batch-processing) is intended for asynchronous work, with a documented discount and completion window; it is not a drop-in latency improvement for interactive requests. Caching (https://platform.claude.com/docs/en/build-with-claude/prompt-caching) depends on reusable prefixes, model-specific minimum lengths and expiry. Its advertised percentage applies to eligible input reads, not the entire task bill.

Avoid copying the dossier's universal 1,024-token threshold or blanket 0.1x cache-read rate: current documentation has model-specific exceptions. Do not install the dossier's proposed `budgetmaxxing` CLI as though it were an established product. No official installation source for that idea was established.

A useful experiment

1. Save 20 representative tasks and expected acceptance criteria. Record correctness, latency and total usage for the current setup.
2. Move stable instructions and reference context before changing request data. Inspect actual cache-read/write fields; don't infer a hit from faster output.
3. Re-run the same tasks with one change at a time. Measure total cost including failures, retries, supervision and review.
4. Batch only work whose callers can wait. Check completion and error records before treating a batch as successful.
5. Try a cheaper executor only on independently verifiable subtasks. Retain the baseline where quality drops.
6. Keep a per-task ledger, rather than guessed savings in prose.

Persistent policy that stays honest

Keep project instructions short: the acceptance bar, where the eval lives, and where usage is recorded. A prompt is guidance; a hook is executable code and needs review. A Stop hook that always exits with a

blocking code can cause a loop, so use a bounded, stateful check rather than the dossier's unconditional example.

Suggested deliverables

An `eval-cases.jsonl`, a `results.csv` containing quality/latency/usage/cost fields, and a report of tradeoffs. Prefer a small measurable change over a complicated router before the baseline exists. This is a proposed experiment, not a measured savings claim.

Reusable project prompt

```
Build Cost-per-accepted-task lab. Offline benchmark harness with optional API adapters; no automatic model downgrade. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Define the acceptance rubric; Record baseline and usage; Compare one optimization at a time; Publish the measured tradeoff table.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://www.instagram.com/reel/DePBKM-zUa3/